Department of Computer Science and Engineering

Semester: Fall 2023

Course Code: CSE 207

Course Title: Data Structures

Lab- 04

Submitted to

Md. Manowarul Islam

Adjunct Faculty, Department of CSE

Submitted by

Swarna Rani Dey

Id No:2022-1-60-340

Section- 7

Task -1

```
#include <stdio.h>

int main()
{
   int option;

   do
   {
      printf("\n");
      printf("1. Insert new node\n");
      printf("2. Insert node at beginning\n");
      printf("3. Insert node at any position\n");
      printf("4. Exit\n");
      printf("Please enter your choice: ");

      scanf("%d", &option);

      switch (option)
      {
      case 1:

         break;
      case 2:

         break;
```

```
        case 3:

            break;

        case 4:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice. Please enter a valid option.\n");
        }

    } while (option != 4);

    return 0;
}
```

OUTPUT

```
Output
/tmp/hntm0e7vrk.o
1. Insert new node
2. Insert node at beginning
3. Insert node at any position
4. Exit
Please enter your choice: 1
1. Insert new node
2. Insert node at beginning
3. Insert node at any position
4. Exit
Please enter your choice:
```

Task-2

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

void insertAtEnd(struct Node** head, int newData) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = newData;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* last = *head;
    while (last->next != NULL) {
        last = last->next;
    }
```

```
    last->next = newNode;
}

void printList(struct Node* node) {
    while (node != NULL) {
        printf("%d ", node->data);
        node = node->next;
    }
    printf("\n");
}

int main() {

    struct Node* head = NULL;

    insertAtEnd(&head, 1);
    insertAtEnd(&head, 2);
    insertAtEnd(&head, 3);

    printf("Linked List: ");
    printList(head);

    return 0;
}
```

OUTPUT

Task-3

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* link;
} *head;

void createList(int n);
void insertAtBeginning(int d);
void display();

int main() {
    int option, n, d;
```

```
while (1) {
    printf("\n");
    printf("1. Create linked list\n");
    printf("2. Insert node at beginning\n");
    printf("3. Insert node at any position\n");
    printf("4. Delete Node from Last Position\n");
    printf("5. Delete Node from Beginning\n");
    printf("6. Delete Node from Any Position\n");
    printf("7. Show list\n");
    printf("8. Exit\n");
    printf("Please enter your choice: ");
    scanf("%d", &option);

    switch (option) {
    case 1:
        printf("Input the number of nodes: ");
        scanf("%d", &n);
        createList(n);
        break;

    case 2:
        printf("Enter data at the beginning node: ");
        scanf("%d", &d);
        insertAtBeginning(d);
        break;
```

```
        case 7:
            display();
            break;

        case 8:
            exit(0);

        default:
            printf("Invalid choice. Please enter a valid option.\n");
        }
    }
    return 0;
}

void createList(int n) {
    struct node *newnode, *temp;
    int data, i;

    head = (struct node*)malloc(sizeof(struct node));
    if (head == NULL) {
        printf("Unable to allocate memory.");
        exit(0);
    }

    printf("Enter the data of node 1: ");
    scanf("%d", &data);
```

```
    head->data = data;
    head->link = NULL;
    temp = head;

    for (i = 2; i <= n; i++) {
        newnode = (struct node*)malloc(sizeof(struct node));
        if (newnode == NULL) {
            printf("Unable to allocate memory.");
            break;
        }

        printf("Enter the data of node %d: ", i);
        scanf("%d", &data);

        newnode->data = data;
        newnode->link = NULL;
        temp->link = newnode;
        temp = temp->link;
    }
}

void insertAtBeginning(int d) {
    struct node* newnode;

    newnode = (struct node*)malloc(sizeof(struct node));
    if (newnode == NULL) {
        printf("Unable to allocate memory.");
```

```
        return;
    }

    newnode->data = d;
    newnode->link = head;
    head = newnode;
}

void display() {
    struct node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    while (temp != NULL) {
        printf("Data = %d\n", temp->data);
        temp = temp->link;
    }
}
```

Task-4

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
    int data;
    struct node* link;
} *head;

void createList(int n);
void insertAtBeginning(int d);
void insertAtAnyPosition(int data, int p);
void display();

int main() {
    int option, n, d, p, dt;

    while (1) {
        printf("\n");
        printf("1. Create linked list\n");
        printf("2. Insert node at beginning\n");
        printf("3. Insert node at any position\n");
        printf("4. Delete Node from Last Position\n");
        printf("5. Delete Node from Beginning\n");
        printf("6. Delete Node from Any Position\n");
        printf("7. Show list\n");
        printf("8. Exit\n");
        printf("Please enter your choice: ");
        scanf("%d", &option);

        switch (option) {
        case 1:
            printf("Input the number of nodes: ");
            scanf("%d", &n);
            createList(n);
            break;

        case 2:
            printf("Enter data at the beginning node: ");
```

```
            scanf("%d", &d);
            insertAtBeginning(d);
            break;

        case 3:
            printf("Enter the position where you want to insert the new node: ");
            scanf("%d", &p);
            printf("Enter data: ");
            scanf("%d", &dt);
            insertAtAnyPosition(dt, p);
            break;

        case 7:
            display();
            break;

        case 8:
            exit(0);

        default:
            printf("Invalid choice. Please enter a valid option.\n");
        }
    }
    return 0;
}

void createList(int n) {
    struct node *newnode, *temp;
    int data, i;

    head = (struct node*)malloc(sizeof(struct node));
    if (head == NULL) {
        printf("Unable to allocate memory.");
        exit(0);
    }
```

```
    printf("Enter the data of node 1: ");
    scanf("%d", &data);

    head->data = data;
    head->link = NULL;
    temp = head;

    for (i = 2; i <= n; i++) {
        newnode = (struct node*)malloc(sizeof(struct node));
        if (newnode == NULL) {
            printf("Unable to allocate memory.");
            break;
        }

        printf("Enter the data of node %d: ", i);
        scanf("%d", &data);

        newnode->data = data;
        newnode->link = NULL;
        temp->link = newnode;
        temp = temp->link;
    }
}

void insertAtBeginning(int d) {
    struct node* newnode;

    newnode = (struct node*)malloc(sizeof(struct node));
    if (newnode == NULL) {
        printf("Unable to allocate memory.");
        return;
    }

    newnode->data = d;
```

```
    newnode->link = head;
    head = newnode;
}

void display() {
    struct node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    while (temp != NULL) {
        printf("Data = %d\n", temp->data);
        temp = temp->link;
    }
}

void insertAtAnyPosition(int data, int p) {
    int i;
    struct node *newnode, *temp;
    newnode = (struct node*)malloc(sizeof(struct node));
    if (newnode == NULL) {
        printf("Unable to allocate memory.");
    } else {
        newnode->data = data;
        newnode->link = NULL;
        temp = head;
        for (i = 2; i <= p - 1; i++) {
            temp = temp->link;
            if (temp == NULL)
                break;
        }
        if (temp != NULL) {
```

```
            newnode->link = temp->link;
            temp->link = newnode;
        } else {
            printf("UNABLE TO INSERT DATA AT THE GIVEN POSITION\n");
        }
    }
}
```

Task-5

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
 int data;
 struct node *link;
} node,*head;
int main()
{
 int option,n,d,p,dt;
 while(1)
 {
 printf("\n");
 printf("1.Create linked list\n");
 printf("2.Insert node at begining\n");
 printf("3.Insert node at any position\n");
```

```
printf("4. Delete Node from Last Position\n");
printf("5. Delete Node from Beginning\n");
printf("6. Delete Node from Any Position\n");
printf("7.show list\n");
printf("Please enter your choice:");
scanf("%d",&option);
switch(option)
{
case 1:
printf(" Input the number of nodes : ");
scanf("%d", &n);
createList(n);
break;
case 2:
printf("Enter data at the begining node: ");
scanf("%d", &d);
InserAtBegining();
break;
case 3:
printf("Enter the position where you want to insert the new node: ");
scanf("%d", &p);
printf("Enter data: ");
scanf("%d", &dt);
InserAtAnyPosition();
break;
case 4:
```

```
 delLastPos();
 case 5:
 display();
 break;
 }
 }
 return 0;
}
void createList(int n)
{
 struct node *newnode, *temp;
 int data, i;
 head = (struct node *)malloc(sizeof(struct node));
 if(head == NULL)
 {
 printf("Unable to allocate memory.");
 exit(0);
 }
 printf("Enter the data of node 1: ");
 scanf("%d", &data);
 head->data = data;
 head->link = NULL;
 temp = head;
 for(i=2; i<=n; i++)
 {
 newnode = (struct node *)malloc(sizeof(struct node));
```

```
 if(newnode == NULL)
 {
 printf("Unable to allocate memory.");
 break;
 }
 printf("Enter the data of node %d: ", i);
 scanf("%d", &data);
 newnode->data = data;
 newnode->link = NULL;
 temp->link = newnode;
 temp = temp->link;
 }
}
void InserAtBegining(struct node *head,int d)
{
 struct node *newnode, *temp;
 head = (struct node *)malloc(sizeof(struct node));
 newnode->data = d;
 newnode->link= head;
 head = newnode;
}
void display()
{
 struct node *temp;
 if(head == NULL)
 {
```

```
 printf("List is empty.");
 return;
 }
 temp = head;
 while(temp != NULL)
 {
 printf("Data = %d\n", temp->data);
 temp = temp->link;
 }
}
void InserAtAnyPosition(int data, int p)
{
 int i;
 struct node *newnode, *temp;
 newnode = (struct node*)malloc(sizeof(struct node));
 if(newnode == NULL)
 {
 printf("Unable to allocate memory.");
 }
 else
 {
 newnode->data = data;
 newnode->link = NULL;
 temp = head;
 for(i=2; i<=p-1; i++)
 {
```

```
 temp = temp->link;
 if(temp == NULL)
 break;
 }
 if(temp != NULL)
 {
 newnode->link = temp->link;
 temp->link = newnode;
 }

 else
 {
 printf("UNABLE TO INSERT DATA AT THE GIVEN POSITION\n");
 }
 }
}
void delLastPos(struct node *head)
{
 if(head==NULL)
 printf("list is already empty");
 else if(head->link==NULL)
 {
 free(head);
 head=NULL;
 }
```

```
else
{
struct node *temp=head;
while(temp->link->link!=NULL)
{
temp=temp->link;
}

free(temp->link);
temp->link=NULL;
}
}
```

Task-6

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *link;
} *head;

void createList(int n);
void InsertAtBeginning(int d);
void InsertAtAnyPosition(int data, int p);
```

```
void delLastPos();
void firstNodeDeletion();
void display();

int main()
{
    int option, n, d, p, dt;
    while (1)
    {
        printf("\n");
        printf("1. Create linked list\n");
        printf("2. Insert node at beginning\n");
        printf("3. Insert node at any position\n");
        printf("4. Delete Node from Last Position\n");
        printf("5. Delete Node from Beginning\n");
        printf("6. Display\n");
        printf("7. Exit\n");
        printf("Please enter your choice: ");
        scanf("%d", &option);

        switch (option)
        {

        case 1:
            printf("Input the number of nodes: ");
            scanf("%d", &n);
            createList(n);
```

```
        break;
    case 2:
        printf("Enter data at the beginning node: ");
        scanf("%d", &d);
        InsertAtBeginning(d);
        break;

    case 3:
        printf("Enter the position where you want to insert the new node: ");
        scanf("%d", &p);
        printf("Enter data: ");
        scanf("%d", &dt);
        InsertAtAnyPosition(dt, p);
        break;

    case 4:
        delLastPos();
        break;

    case 5:
        firstNodeDeletion();
        break;

    case 6:
        display();
        break;
```

```
        case 7:
            exit(0);
        default:
            printf("Invalid choice! Please enter a valid option.\n");
        }
    }
    return 0;
}

void createList(int n)
{
    struct node *newnode, *temp;
    int data, i;
    head = (struct node *)malloc(sizeof(struct node));
    if (head == NULL)
    {
        printf("Unable to allocate memory.");
        exit(0);
    }
    printf("Enter the data of node 1: ");
    scanf("%d", &data);
    head->data = data;
    head->link = NULL;
    temp = head;
    for (i = 2; i <= n; i++)
    {
        newnode = (struct node *)malloc(sizeof(struct node));
```

```
        if (newnode == NULL)
        {
            printf("Unable to allocate memory.");
            break;
        }
        printf("Enter the data of node %d: ", i);
        scanf("%d", &data);
        newnode->data = data;
        newnode->link = NULL;
        temp->link = newnode;
        temp = temp->link;
    }
}

void InsertAtBeginning(int d)
{
    struct node *newnode;
    newnode = (struct node *)malloc(sizeof(struct node));
    newnode->data = d;
    newnode->link = head;
    head = newnode;
}

void display()
{
    struct node *temp;
    if (head == NULL)
```

```
    {
        printf("List is empty.");
        return;
    }
    temp = head;
    while (temp != NULL)
    {
        printf("Data = %d\n", temp->data);
        temp = temp->link;
    }
}

void InsertAtAnyPosition(int data, int p)
{
    int i;
    struct node *newnode, *temp;
    newnode = (struct node *)malloc(sizeof(struct node));
    if (newnode == NULL)
    {
        printf("Unable to allocate memory.");
    }
    else
    {
        newnode->data = data;
        newnode->link = NULL;
        temp = head;
        for (i = 2; i <= p - 1; i++)
```

```
        {
            temp = temp->link;
            if (temp == NULL)
                break;
        }
        if (temp != NULL)
        {
            newnode->link = temp->link;
            temp->link = newnode;
        }
        else
        {
            printf("UNABLE TO INSERT DATA AT THE GIVEN POSITION\n");
        }
    }
}

void delLastPos()
{
    if (head == NULL)
        printf("List is already empty");
    else if (head->link == NULL)
    {
        free(head);
        head = NULL;
    }
    else
```

```
    {
        struct node *temp = head;
        while (temp->link->link != NULL)
        {
            temp = temp->link;
        }
        free(temp->link);
        temp->link = NULL;
    }
}

void firstNodeDeletion()
{
    struct node *Delnode;
    if (head == NULL)
    {
        printf("There are no nodes in the list.");
    }
    else
    {
        Delnode = head;
        head = head->link;
        printf("\nDeleted Data: %d\n", Delnode->data);
        free(Delnode);
    }
}
```

Task-7

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
 int data;
 struct node *link;
} *node,*head;
int main()
{
 int option,n,d,p,dt;
 while(1)
 {
 printf("\n");
 printf("1.Create linked list\n");
 printf("2.Insert node at begining\n");
 printf("3.Insert node at any position\n");
 printf("4. Delete Node from Last Position\n");
 printf("5. Delete Node from Beginning\n");
 printf("6. Delete Node from Any Position\n");
 printf("7.show list\n");
 printf("Please enter your choice:");
 scanf("%d",&option);
 switch(option)
 {
 case 1:
 printf(" Input the number of nodes : ");
 scanf("%d", &n);
 createList(n);
 break;
 case 2:
 printf("Enter data at the begining node: ");
 scanf("%d", &d);
```

```
 InserAtBegining();
 break;
 case 3:
 printf("Enter the position where you want to insert the new node: ");
 scanf("%d", &p);
 printf("Enter data: ");
 scanf("%d", &dt);
 InserAtAnyPosition();
 break;
 case 4:
 delLastPos();
 break;
 case 5:
 firstNodeDeletion();
 break;
 case 6:
 printf("position of the node to delete: ");
 scanf("%d", &p);
 deleteAnyPos();
 break;
 case 7:
 display();
 break;
 }
 }
 return 0;
}
void createList(int n)
{
 struct node *newnode, *temp;
 int data, i;
 head = (struct node *)malloc(sizeof(struct node));
 if(head == NULL)
 {
 printf("Unable to allocate memory.");
```

```
 exit(0);
 }
 printf("Enter the data of node 1: ");
 scanf("%d", &data);
 head->data = data;
 head->link = NULL;
 temp = head;
 for(i=2; i<=n; i++)
 {
 newnode = (struct node *)malloc(sizeof(struct node));
 if(newnode == NULL)
 {
 printf("Unable to allocate memory.");
 break;
 }
 printf("Enter the data of node %d: ", i);
 scanf("%d", &data);
 newnode->data = data;
 newnode->link = NULL;
 temp->link = newnode;
 temp = temp->link;
 }
}
void InserAtBegining(struct node *head,int d)
{
 struct node *newnode, *temp;
 head = (struct node *)malloc(sizeof(struct node));
 newnode->data = d;
 newnode->link= head;
 head = newnode;
}
void display()
{
 struct node *temp;
 if(head == NULL)
```

```
 {
 printf("List is empty.");
 return;
 }
 temp = head;
 while(temp != NULL)
 {
 printf("Data = %d\n", temp->data);
 temp = temp->link;
 }
}
void InserAtAnyPosition(int data, int p)
{
 int i;
 struct node *newnode, *temp;
 newnode = (struct node*)malloc(sizeof(struct node));
 if(newnode == NULL)
 {
 printf("Unable to allocate memory.");
 }
 else
 {
 newnode->data = data;
 newnode->link = NULL;
 temp = head;
 for(i=2; i<=p-1; i++)
 {
 temp = temp->link;
 if(temp == NULL)
 break;
 }
 if(temp != NULL)
 {
 newnode->link = temp->link;
 temp->link = newnode;
```

```
 }
 else
 {
 printf("UNABLE TO INSERT DATA AT THE GIVEN POSITION\n");
 }
 }
}
void delLastPos(struct node *head)
{
 if(head==NULL)
 printf("list is already empty");
 else if(head->link==NULL)
 {
 free(head);
 head=NULL;
 }
 else
 {
 struct node *temp=head;
 while(temp->link->link!=NULL)
 {
 temp=temp->link;
 }
 free(temp->link);
 temp->link=NULL;
 }
}
void firstNodeDeletion()
{
 struct node *Delnode;
 if(node = NULL)
 {
 printf(" There are no node in the list.");
 }
 else
```

```
 {
 Delnode = node;
 node = node->link;
 printf("\n deleted Data : %d\n", Delnode->data);
 free(Delnode);
 }
}
void deleteAnyPos(int pos)
{
 struct node *temp = head;
 int i;
 if(pos==0)
 {
 printf("\nElement deleted is : %d\n",temp->data);
 head=head->link;
 temp->link=NULL;
 free(temp);
 }
 else
 {
 for(i=0; i<pos-1; i++)
 {
 temp=temp->link;
 }
 struct node *del =temp->link;
 temp->link=temp->link->link;
 printf("\nElement deleted is : %d\n",del->data);
 del->link=NULL;
 free(del);
 }
}
```

Task-8

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *link;
} *head;

void createList(int n);
void InsertAtBeginning(int d);
void InsertAtAnyPosition(int data, int p);
void delLastPos();
void firstNodeDeletion();
void deleteAnyPos(int pos);
void reverseList();
void display();

int main()
{
   int option, n, d, p, dt;
   while (1)
   {
      printf("\n");
      printf("1. Create linked list\n");
      printf("2. Insert node at beginning\n");
      printf("3. Insert node at any position\n");
      printf("4. Delete Node from Last Position\n");
      printf("5. Delete Node from Beginning\n");
      printf("6. Delete Node from Any Position\n");
      printf("7. Reverse Linked List\n");
      printf("8. Remove Duplicate Data from Linked List\n");
```

```
printf("9. Show list\n");
printf("10. Exit\n");
printf("Please enter your choice: ");
scanf("%d", &option);

switch (option)
{
case 1:
   printf("Input the number of nodes: ");
   scanf("%d", &n);
   createList(n);
   break;
case 2:
   printf("Enter data at the beginning node: ");
   scanf("%d", &d);
   InsertAtBeginning(d);
   break;
case 3:
   printf("Enter the position where you want to insert the new node: ");
   scanf("%d", &p);
   printf("Enter data: ");
   scanf("%d", &dt);
   InsertAtAnyPosition(dt, p);
   break;
case 4:
   delLastPos();
   break;
case 5:
   firstNodeDeletion();
   break;
case 6:
   printf("Position of the node to delete: ");
   scanf("%d", &p);
   deleteAnyPos(p);
   break;
```

```
        case 7:
            reverseList();
            break;
        case 8:
            break;

        case 9:
            display();
            break;

        case 10:
            exit(0);
        default:
            printf("Invalid choice! Please enter a valid option.\n");
        }
    }
    return 0;
}
```

Lab Task-9

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
 int data;
 struct node *link;
} *node,*head;
int main()
{
 int option,n,d,p,dt;
 while(1)
```

```
 {
 printf("\n");
 printf("1.Create linked list\n");
 printf("2.Insert node at begining\n");
 printf("3.Insert node at any position\n");
 printf("4. Delete Node from Last Position\n");
 printf("5. Delete Node from Beginning\n");
 printf("6. Delete Node from Any Position\n");
printf("7. Reverse Linked List\n");
 printf("8. Remove Duplicate Data from Linked List\n");
 printf("9.show list\n");
 printf("Please enter your choice:");
 scanf("%d",&option);
 switch(option)
 {
 case 1:
 printf(" Input the number of nodes : ");
 scanf("%d", &n);
 createList(n);
 break;
 case 2:
 printf("Enter data at the begining node: ");
 scanf("%d", &d);
 InserAtBegining();
 break;
 case 3:
 printf("Enter the position where you want to insert the new node: ");
```

```
scanf("%d", &p);
printf("Enter data: ");
scanf("%d", &dt);
InserAtAnyPosition();
break;
case 4:
delLastPos();
break;
case 5:
firstNodeDeletion();
break;
case 6:
printf("position of the node to delete: ");
scanf("%d", &p);
deleteAnyPos();
break;
case 7:
reverseList();
break;
case 8:
removeDuplicate();
break;
case 9:
display();
break;
}
}
```

```
 return 0;
}
void createList(int n)
{
 struct node *newnode, *temp;
 int data, i;
 head = (struct node *)malloc(sizeof(struct node));
 if(head == NULL)
 {
 printf("Unable to allocate memory.");
 exit(0);
 }
 printf("Enter the data of node 1: ");
 scanf("%d", &data);
 head->data = data;
 head->link = NULL;
 temp = head;
 for(i=2; i<=n; i++)
 {
 newnode = (struct node *)malloc(sizeof(struct node));
 if(newnode == NULL)
 {
 printf("Unable to allocate memory.");
 break;
 }
 printf("Enter the data of node %d: ", i);
 scanf("%d", &data);
```

```
 newnode->data = data;
 newnode->link = NULL;
 temp->link = newnode;
 temp = temp->link;
 }
}
void InserAtBegining(struct node *head,int d)
{
 struct node *newnode, *temp;
 head = (struct node *)malloc(sizeof(struct node));
 newnode->data = d;
 newnode->link= head;
 head = newnode;
}
void display()
{
 struct node *temp;
 if(head == NULL)
 {
 printf("List is empty.");
 return;
 }
 temp = head;
 while(temp != NULL)
 {
 printf("Data = %d\n", temp->data);
 temp = temp->link;
```

```
 }
}
void InserAtAnyPosition(int data, int p)
{
 int i;
 struct node *newnode, *temp;
 newnode = (struct node*)malloc(sizeof(struct node));
 if(newnode == NULL)
 {
 printf("Unable to allocate memory.");
 }
 else
 {
 newnode->data = data;
 newnode->link = NULL;
 temp = head;
 for(i=2; i<=p-1; i++)
 {
 temp = temp->link;
 if(temp == NULL)
 break;
 }
 if(temp != NULL)
 {
 newnode->link = temp->link;
 temp->link = newnode;
 }
```

```
 else
 {
 printf("UNABLE TO INSERT DATA AT THE GIVEN POSITION\n");
 }
 }
}
void delLastPos(struct node *head)
{
 if(head==NULL)
 printf("list is already empty");
 else if(head->link==NULL)
 {
 free(head);
 head=NULL;
 }
 else
 {
 struct node *temp=head;
 while(temp->link->link!=NULL)
 {
 temp=temp->link;
 }
 free(temp->link);
 temp->link=NULL;
 }
}
void firstNodeDeletion()
```

```
{
 struct node *Delnode;
 if(node = NULL)
 {
 printf(" There are no node in the list.");
 }
 else
 {
 Delnode = node;
 node = node->link;
 printf("\n deleted Data : %d\n", Delnode->data);
 free(Delnode);
 }
}
void deleteAnyPos(int pos)
{
 struct node *temp = head;
 int i;
 if(pos==0)
 {
 printf("\nElement deleted is : %d\n",temp->data);
 head=head->link;
 temp->link=NULL;
 free(temp);
 }
 else
 {
```

```
 for(i=0; i<pos-1; i++)
 {
 temp=temp->link;
 }
 struct node *del =temp->link;
 temp->link=temp->link->link;
 printf("\nElement deleted is : %d\n",del->data);
 del->link=NULL;
 free(del);
 }
}
void reverseList()
{
 struct node *prenode, *curnode;
 if(head != NULL)
 {
 prenode = head;
 curnode = head->link;
 head = head->link;
 prenode->link = NULL;
 while(head != NULL)
 {
 head = head->link;
 curnode->link = prenode;
 prenode = curnode;
 curnode = head;
 }
```

```
head = prenode;
 printf("SUCCESSFULLY REVERSED LIST\n");
 }
}
void removeDuplicate() {
 struct node *cur = head, *in = NULL, *temp = NULL;
 if(head == NULL) {
 return;
 }
 else {
 while(cur != NULL){
 temp = cur;
 in = cur->link;
 while(in != NULL) {
 if(cur->data == in->data) {
 temp->link = in->link;
 }
 else {
 temp = in;
 }
 in = in->link;
 }
 cur = cur->link;
 }
 }
}
```

OUTPUT

```
Output
/tmp/VxvjNrCIQt.o
1.Create linked list
2.Insert node at begining
3.Insert node at any position
4. Delete Node from Last Position
5. Delete Node from Beginning
6. Delete Node from Any Position
7. Reverse Linked List
8. Remove Duplicate Data from Linked List
9.show list
Please enter your choice:1
Input the number of nodes : 6
Enter the data of node 1: 1
Enter the data of node 2: 2
Enter the data of node 3: 1
Enter the data of node 4: 1
Enter the data of node 5: 2
Enter the data of node 6: 3
1.Create linked list
2.Insert node at begining
3.Insert node at any position
4. Delete Node from Last Position
5. Delete Node from Beginning
6. Delete Node from Any Position
7. Reverse Linked List
8. Remove Duplicate Data from Linked List
9.show list
Please enter your choice:8
1.Create linked list
2.Insert node at begining
3.Insert node at any position
4. Delete Node from Last Position
5. Delete Node from Beginning
6. Delete Node from Any Position
7. Reverse Linked List
8. Remove Duplicate Data from Linked List
9.show list
Please enter your choice:9
Data = 1
Data = 2
Data = 3

1.Create linked list
2.Insert node at begining
3.Insert node at any position
4. Delete Node from Last Position
5. Delete Node from Beginning
6. Delete Node from Any Position
7. Reverse Linked List
8. Remove Duplicate Data from Linked List
9.show list
```